

What is Apache Airflow?

Apache Airflow is an open-source workflow management and orchestration platform designed to automate, schedule, and monitor complex workflows and data pipelines.

Apache Airflow enables organizations to define workflows programmatically using Python code, where each workflow consists of multiple tasks connected in a logical sequence. It ensures that tasks execute in the correct order, at the correct time, and with proper dependency management.

Airflow is mainly used in:

- Data Engineering
- ETL Pipelines
- Data Warehousing
- Machine Learning Pipelines
- Cloud Automation
- Batch Processing Systems

It is widely adopted because it provides:

- Workflow scheduling
- Task dependency handling
- Monitoring dashboards
- Automatic retries
- Logging and alerting
- Scalable execution

Apache Airflow can be formally defined as:

“A distributed workflow orchestration platform used to author, schedule, execute, and monitor workflows as Directed Acyclic Graphs (DAGs) using Python.”

Distributed Workflow Orchestration Platform

Distributed

Airflow can run tasks across multiple machines and servers instead of a single system.

Workflow

A workflow is a sequence of connected tasks executed in a specific order.

Orchestration

Orchestration means coordinating and managing multiple tasks automatically.

Platform

Airflow provides a complete environment including:

- Scheduler
- Web UI
- Task executor
- Monitoring tools
- Logging system

What Problem Does Airflow Solve?

In organizations, many tasks need to run repeatedly and in sequence.

For example:

1. Extract customer data from databases
2. Clean the data
3. Transform the data
4. Load data into a warehouse
5. Generate reports
6. Send notification emails

Doing this manually every day is inefficient and error-prone.

Apache Airflow automates this entire process.

Real-Time Example

Suppose an e-commerce company wants a daily sales pipeline.

Every day at 1 AM:

- Sales data is extracted from MySQL
- Data is cleaned
- Revenue calculations are performed
- Results are loaded into Snowflake
- Dashboards are refreshed

Airflow schedules and manages all these activities automatically.

What is a DAG in Airflow?

A DAG (Directed Acyclic Graph) is a collection of tasks organized in a way that defines:

- The execution order
- Dependencies between tasks
- Workflow structure

It is the core building block of Apache Airflow workflows.

DAG

Directed

Tasks move in a specific direction.

Example:

Extract → Transform → Load

Acyclic

Tasks cannot form loops.

Wrong:

$A \rightarrow B \rightarrow A$

Correct:

$A \rightarrow B \rightarrow C$

Graph

A structure made of connected tasks.

Task

A task is:

“An individual unit of work executed as part of a workflow.”

Examples:

- Running SQL query
- Reading CSV file
- Sending email
- Executing Spark job
- Loading data into database

Operator

An Operator is:

“A predefined template that determines what action a task performs.”

Operators define the behavior of tasks.

Examples:

- PythonOperator
- BashOperator
- EmailOperator
- SQL Operator

Scheduler

The Scheduler is:

“The Airflow component responsible for monitoring DAGs, identifying runnable tasks, and triggering task execution based on schedules and dependencies.”

The Scheduler continuously checks:

- Time schedules
- DAG definitions
- Task completion status

Executor

The Executor is:

“The component responsible for executing tasks submitted by the Scheduler.”

It determines:

- Where tasks run
- How tasks run
- Parallel execution handling

Metadata Database

The Metadata Database is:

“The central storage system used by Airflow to maintain workflow state, task history, logs, scheduling details, and user metadata.”

Common databases:

- PostgreSQL
- MySQL

Web Server

The Web Server is:

“The component that provides Airflow’s graphical user interface for monitoring, managing, and visualizing workflows and task execution.”

Using the UI, users can:

- Trigger DAGs
- Pause workflows
- View logs
- Monitor failures
- Track execution history

Advantages of Apache Airflow

1. Workflow Automation

Airflow automates repetitive and scheduled processes without manual intervention.

Example

Daily ETL jobs run automatically every midnight.

2. Dependency Management

Airflow ensures tasks execute in the correct order.

Example

Data transformation starts only after extraction completes.

```
extract >> transform >> load
```

3. Scalability

Airflow supports distributed execution and large-scale workflows across multiple servers.

4. Monitoring and Logging

Provides centralized monitoring, task logs, execution history, and failure tracking.

5. Retry and Failure Handling

Automatically retries failed tasks based on configured retry policies.

```
retries=3
```

6. Python Integration

Airflow workflows are written entirely in Python, making them flexible and customizable.

Airflow Architecture

Apache Airflow architecture consists of several interconnected components working together.

1. DAG Files

Python scripts containing workflow definitions.

Example:

daily_sales_pipeline.py

2. Scheduler

Reads DAGs and schedules tasks.

3. Executor

Executes scheduled tasks.

4. Workers

Perform actual task execution.

5. Metadata Database

Stores execution information.

6. Web Server

Provides graphical interface.

Architecture Flow

Developer Writes DAG



DAG Stored in DAG Folder



Scheduler Reads DAG



Scheduler Sends Tasks to Executor



Executor Assigns Tasks to Workers



Workers Execute Tasks



Metadata Stored in Database



Web UI Displays Status and Logs

Apache Airflow DAG Example

```
from airflow import DAG

from airflow.operators.python import PythonOperator

from datetime import datetime

def extract():

    print("Extracting data")

def transform():

    print("Transforming data")

def load():

    print("Loading data")

with DAG(

    dag_id='etl_pipeline', start_date=datetime(2025,1,1), schedule_interval='@daily',

    catchup=False

) as dag:

    extract_task = PythonOperator(task_id='extract', python_callable=extract)

    transform_task = PythonOperator(task_id='transform', python_callable=transform )

    load_task = PythonOperator( task_id='load', python_callable=load )

    extract_task >> transform_task >> load_task
```

SQL Queries Used in Airflow Pipelines

Extract Data

```
SELECT *

FROM sales

WHERE order_date = CURRENT_DATE;
```

Revenue Aggregation

```
SELECT product_id,  
       SUM(amount) AS total_revenue  
FROM sales  
GROUP BY product_id;
```

Load Process

```
INSERT INTO sales_summary  
SELECT product_id,  
       SUM(amount)  
FROM sales  
GROUP BY product_id;
```

Conclusion

Apache Airflow is a powerful workflow orchestration platform used to automate and manage complex workflows and data pipelines.

It provides:

- Scheduling
- Dependency management
- Monitoring
- Scalability
- Automation
- Fault tolerance

Because of these capabilities, Airflow has become one of the most widely used orchestration tools in modern Data Engineering and cloud-based data platforms.